

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

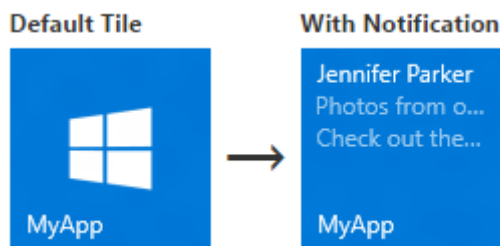
Send a local tile notification

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Primary app tiles in Windows 10 are defined in your app manifest, while secondary tiles are programmatically created and defined by your app code. This article describes how to send a local tile notification to a primary tile and a secondary tile using adaptive tile templates. (A local notification is one that's sent from app code as opposed to one that's pushed or pulled from a web server.)



ⓘ Note

Learn about [creating adaptive tiles](#) and [tile content schema](#).

Install the NuGet package

We recommend installing the [Notifications library NuGet package](#) [↗](#), which simplifies things by generating tile payloads with objects instead of raw XML.

The inline code examples in this article are for C# using the Notifications library. (If you'd prefer to create your own XML, you can find code examples without the Notifications library toward the end of the article.)

Add namespace declarations

To access the tile APIs, include the [Windows.UI.Notifications](#) namespace. We also recommend including the [Microsoft.Toolkit.Uwp.Notifications](#) namespace so that you

can take advantage of our tile helper APIs (you must install the [Notifications library](#) NuGet package to access these APIs).

C#

```
using Windows.UI.Notifications;
using Microsoft.Toolkit.Uwp.Notifications; // Notifications library
```

Create the notification content

In Windows 10, tile payloads are defined using adaptive tile templates, which allow you to create custom visual layouts for your notifications. (To learn what's possible with adaptive tiles, see [Create adaptive tiles](#).)

This code example creates adaptive tile content for medium and wide tiles.

C#

```
// In a real app, these would be initialized with actual data
string from = "Jennifer Parker";
string subject = "Photos from our trip";
string body = "Check out these awesome photos I took while in New Zealand!";

// Construct the tile content
TileContent content = new TileContent()
{
    Visual = new TileVisual()
    {
        TileMedium = new TileBinding()
        {
            Content = new TileBindingContentAdaptive()
            {
                Children =
                {
                    new AdaptiveText()
                    {
                        Text = from
                    },

                    new AdaptiveText()
                    {
                        Text = subject,
                        HintStyle = AdaptiveTextStyle.CaptionSubtle
                    },

                    new AdaptiveText()
                    {
                        Text = body,
                        HintStyle = AdaptiveTextStyle.CaptionSubtle
                    }
                }
            }
        }
    }
}
```

```

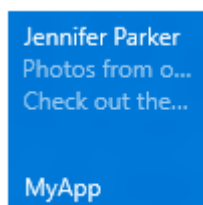
    }
    },
    TileWide = new TileBinding()
    {
        Content = new TileBindingContentAdaptive()
        {
            Children =
            {
                new AdaptiveText()
                {
                    Text = from,
                    HintStyle = AdaptiveTextStyle.Subtitle
                },

                new AdaptiveText()
                {
                    Text = subject,
                    HintStyle = AdaptiveTextStyle.CaptionSubtle
                },

                new AdaptiveText()
                {
                    Text = body,
                    HintStyle = AdaptiveTextStyle.CaptionSubtle
                }
            }
        }
    }
};

```

The notification content looks like the following when displayed on a medium tile:



Create the notification

Once you have your notification content, you'll need to create a new **TileNotification**. The **TileNotification** constructor takes a Windows Runtime **XmlDocument** object, which you can obtain from the **TileContent.GetXml** method if you're using the [Notifications library](#).

This code example creates a notification for a new tile.

C#

```
// Create the tile notification  
var notification = new TileNotification(content.GetXml());
```

Set an expiration time for the notification (optional)

By default, local tile and badge notifications don't expire, while push, periodic, and scheduled notifications expire after three days. Because tile content shouldn't persist longer than necessary, it's a best practice to set an expiration time that makes sense for your app, especially on local tile and badge notifications.

This code example creates a notification that expires and will be removed from the tile after ten minutes.

C#

```
tileNotification.ExpirationTime = DateTimeOffset.UtcNow.AddMinutes(10);
```

Send the notification

Although locally sending a tile notification is simple, sending the notification to a primary or secondary tile is a bit different.

Primary tile

To send a notification to a primary tile, use the [TileUpdateManager](#) to create a tile updater for the primary tile, and send the notification by calling "Update". Regardless of whether it's visible, your app's primary tile always exists, so you can send notifications to it even when it's not pinned. If the user pins your primary tile later, the notifications that you sent will appear then.

This code example sends a notification to a primary tile.

C#

```
// Send the notification to the primary tile  
TileUpdateManager.CreateTileUpdaterForApplication().Update(notification);
```

Secondary tile

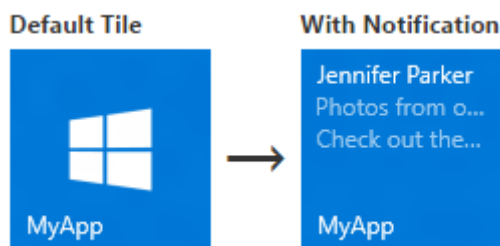
To send a notification to a secondary tile, first make sure that the secondary tile exists. If you try to create a tile updater for a secondary tile that doesn't exist (for example, if the user unpinned the secondary tile), an exception will be thrown. You can use [SecondaryTile.Exists\(tileId\)](#) to discover if your secondary tile is pinned, and then create a tile updater for the secondary tile and send the notification.

This code example sends a notification to a secondary tile.

C#

```
// If the secondary tile is pinned
if (SecondaryTile.Exists("MySecondaryTile"))
{
    // Get its updater
    var updater =
    TileUpdateManager.CreateTileUpdaterForSecondaryTile("MySecondaryTile");

    // And send the notification
    updater.Update(notification);
}
```



Clear notifications on the tile (optional)

In most cases, you should clear a notification once the user has interacted with that content. For example, when the user launches your app, you might want to clear all the notifications from the tile. If your notifications are time-bound, we recommend that you set an expiration time on the notification instead of explicitly clearing the notification.

This code example clears the tile notification for the primary tile. You can do the same for secondary tiles by creating a tile updater for the secondary tile.

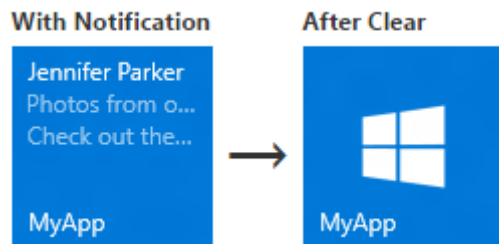
C#

```
TileUpdateManager.CreateTileUpdaterForApplication().Clear();
```

For a tile with the notification queue enabled and notifications in the queue, calling the `Clear` method empties the queue. You can't, however, clear a notification via your app's

server; only the local app code can clear notifications.

Periodic or push notifications can only add new notifications or replace existing notifications. A local call to the `Clear` method will clear the tile whether or not the notifications themselves came via push, periodic, or local. Scheduled notifications that haven't yet appeared are not cleared by this method.



Next steps

Using the notification queue

Now that you have done your first tile update, you can expand the functionality of the tile by enabling a [notification queue](#).

Other notification delivery methods

This article shows you how to send the tile update as a notification. To explore other methods of notification delivery, including scheduled, periodic, and push, see [Delivering notifications](#).

XmlEncode delivery method

If you're not using the [Notifications library](#), this notification delivery method is another alternative.

C#

```
public string XmlEncode(string text)
{
    StringBuilder builder = new StringBuilder();
    using (var writer = XmlWriter.Create(builder))
    {
        writer.WriteString(text);
    }

    return builder.ToString();
}
```

Code examples without Notifications library

If you prefer to work with raw XML instead of the [Notifications library](#) NuGet package, use these alternate code examples to first three examples provided in this article. The rest of the code examples can be used either with the [Notifications library](#) or with raw XML.

Add namespace declarations

C#

```
using Windows.UI.Notifications;  
using Windows.Data.Xml.Dom;
```

Create the notification content

C#

```
// In a real app, these would be initialized with actual data  
string from = "Jennifer Parker";  
string subject = "Photos from our trip";  
string body = "Check out these awesome photos I took while in New Zealand!";  
  
// TODO - all values need to be XML escaped  
  
// Construct the tile content as a string  
string content = @"  
<tile>  
    <visual>  
  
        <binding template='TileMedium'>  
            <text>{from}</text>  
            <text hint-style='captionSubtle'>{subject}</text>  
            <text hint-style='captionSubtle'>{body}</text>  
        </binding>  
  
        <binding template='TileWide'>  
            <text hint-style='subtitle'>{from}</text>  
            <text hint-style='captionSubtle'>{subject}</text>  
            <text hint-style='captionSubtle'>{body}</text>  
        </binding>  
  
    </visual>  
</tile>";
```

Create the notification

C#

```
// Load the string into an XmlDocument
XmlDocument doc = new XmlDocument();
doc.LoadXml(content);

// Then create the tile notification
var notification = new TileNotification(doc);
```

Related topics

- [Create adaptive tiles](#)
- [Tile content schema](#)
- [Notifications library](#) 
- [Full code sample on GitHub](#) 
- [Windows.UI.Notifications namespace](#)
- [How to use the notification queue \(XAML\)](#)
- [Delivering notifications](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Chaseable tile notifications

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Chaseable tile notifications let you determine which tile notifications your app's Live Tile was displaying when the user clicked the tile. For example, a news app could use this feature to determine which news story its Live Tile was displaying when the user launched it; it could then ensure that the story is prominently displayed so that the user can find it.

ⓘ Important

Requires Anniversary Update: To use chaseable tile notifications with C#, C++, or VB-based UWP apps, you must target SDK 14393 and be running build 14393 or later.

Important APIs: [LaunchActivatedEventArgs.TileActivatedInfo](#) property, [TileActivatedInfo](#) class

How it works

To enable chaseable tile notifications, you use the **Arguments** property on the tile notification payload, similar to the launch property on the toast notification payload, to embed info about the content in the tile notification.

When your app is launched via the Live Tile, the system returns a list of arguments from the current/recently displayed tile notifications.

When to use chaseable tile notifications

Chaseable tile notifications are typically used when you're using the notification queue on your Live Tile (which means you are cycling through up to 5 different notifications). They're also beneficial when the content on your Live Tile is potentially out of sync with

the latest content in the app. For example, the News app refreshes its Live Tile every 30 minutes, but when the app is launched, it loads the latest news (which may not include something that was on the tile from the last polling interval). When this happens, the user might get frustrated about not being able to find the story they saw on their Live Tile. That's where chaseable tile notifications can help, by allowing you to make sure that what the user saw on their Tile is easily discoverable.

What to do with a chaseable tile notifications

The most important thing to note is that in most scenarios, **you should NOT directly navigate to the specific notification** that was on the Tile when the user clicked it. Your Live Tile is used as an entry point to your application. There can be two scenarios when a user clicks your Live Tile: (1) they wanted to launch your app normally, or (2) they wanted to see more information about a specific notification that was on the Live Tile. Since there's no way for the user to explicitly say which behavior they want, the ideal experience is to **launch your app normally, while making sure that the notification the user saw is easily discoverable**.

For example, clicking the MSN News app's Live Tile launches the app normally: it displays the home page, or whichever article the user was previously reading. However, on the home page, the app ensures that the story from the Live Tile is easily discoverable. That way, both scenarios are supported: the scenario where you simply want to launch/resume the app, and the scenario where you want to view the specific story.

How to include the Arguments property in your tile notification payload

In a notification payload, the arguments property enables your app to provide data you can use to later identify the notification. For example, your arguments might include the story's id, so that when launched, you can retrieve and display the story. The property accepts a string, which can be serialized however you like (query string, JSON, etc), but we typically recommend query string format, since it's lightweight and XML-encodes nicely.

The property can be set on both the **TileVisual** and the **TileBinding** elements, and will cascade down. If you want the same arguments on every tile size, simply set the arguments on the **TileVisual**. If you need specific arguments for specific tile sizes, you can set the arguments on individual **TileBinding** elements.

This example creates a notification payload that uses the arguments property so that notification can be identified later.

C#

```
// Uses the following NuGet packages
// - Microsoft.Toolkit.Uwp.Notifications
// - QueryString.NET

TileContent content = new TileContent()
{
    Visual = new TileVisual()
    {
        // These arguments cascade down to Medium and Wide
        Arguments = new QueryString()
        {
            { "action", "storyClicked" },
            { "story", "201c9b1" }
        }.ToString(),

        // Medium tile
        TileMedium = new TileBinding()
        {
            Content = new TileBindingContentAdaptive()
            {
                // Omitted
            }
        },

        // Wide tile is same as Medium
        TileWide = new TileBinding() { /* Omitted */ },

        // Large tile is an aggregate of multiple stories
        // and therefore needs different arguments
        TileLarge = new TileBinding()
        {
            Arguments = new QueryString()
            {
                { "action", "storiesClicked" },
                { "story", "43f939ag" },
                { "story", "201c9b1" },
                { "story", "d9481ca" }
            }.ToString(),

            Content = new TileBindingContentAdaptive() { /* Omitted */ }
        }
    }
};
```

How to check for the arguments property when your app launches

Most apps have an App.xaml.cs file that contains an override for the [OnLaunched](#) method. As its name suggests, your app calls this method when it's launched. It takes a single argument, a [LaunchActivatedEventArgs](#) object.

The [LaunchActivatedEventArgs](#) object has a property that enables chaseable notifications: the [TileActivatedInfo](#) property, which provides access to a [TileActivatedInfo](#) object. When the user launches your app from its tile (rather than the app list, search, or any other entry point), your app initializes this property.

The [TileActivatedInfo](#) object contains a property called [RecentlyShownNotifications](#), which contains a list of notifications that have been shown on the tile within the last 15 minutes. The first item in the list represents the notification currently on the tile, and the subsequent items represent the notifications that the user saw before the current one. If your tile has been cleared, this list is empty.

Each [ShownTileNotification](#) has an [Arguments](#) property. The [Arguments](#) property will be initialized with the arguments string from your tile notification payload, or null if your payload didn't include the arguments string.

C#

```
protected override void OnLaunched(LaunchActivatedEventArgs args)
{
    // If the API is present (doesn't exist on 10240 and 10586)
    if
    (ApiInformation.IsPropertyPresent(typeof(LaunchActivatedEventArgs).FullName,
    nameof(LaunchActivatedEventArgs.TileActivatedInfo)))
    {
        // If clicked on from tile
        if (args.TileActivatedInfo != null)
        {
            // If tile notification(s) were present
            if (args.TileActivatedInfo.RecentlyShownNotifications.Count > 0)
            {
                // Get arguments from the notifications that were recently
                displayed
                string[] allArgs =
                args.TileActivatedInfo.RecentlyShownNotifications
                .Select(i => i.Arguments)
                .ToArray();

                // TODO: Highlight each story in the app
            }
        }
    }
}
```

```
// TODO: Initialize app  
}
```

Accessing OnLaunched from desktop applications

Desktop apps (like WPF, etc) using the [Desktop Bridge](#), can use chaseable tiles too! The only difference is accessing the OnLaunched arguments. Note that you first must [package your app with the Desktop Bridge](#).

❗ Important

Requires October 2018 Update: To use the `AppInstance.GetActivatedEventArgs()` API, you must target SDK 17763 and be running build 17763 or later.

For desktop applications, to access the launch arguments, do the following...

C#

```
static void Main()  
{  
    Application.EnableVisualStyles();  
    Application.SetCompatibleTextRenderingDefault(false);  
  
    // API only available on build 17763 or later  
    var args = AppInstance.GetActivatedEventArgs();  
    switch (args.Kind)  
    {  
        case ActivationKind.Launch:  
  
            var launchArgs = args as LaunchActivatedEventArgs;  
  
            // If clicked on from tile  
            if (launchArgs.TileActivatedInfo != null)  
            {  
                // If tile notification(s) were present  
                if  
(launchArgs.TileActivatedInfo.RecentlyShownNotifications.Count > 0)  
                {  
                    // Get arguments from the notifications that were  
                    recently displayed  
                    string[] allTileArgs =  
launchArgs.TileActivatedInfo.RecentlyShownNotifications  
                        .Select(i => i.Arguments)  
                        .ToArray();  
  
                    // TODO: Highlight each story in the app
```

```
    }  
}  
  
break;
```

Raw XML example

If you're using raw XML instead of the Notifications library, here's the XML.

XML

```
<tile>  
  <visual arguments="action=storyClicked&story=201c9b1">  
  
    <binding template="TileMedium">  
  
      <text>Kitten learns how to drive a car...</text>  
      ... (omitted)  
  
    </binding>  
  
    <binding template="TileWide">  
      ... (same as Medium)  
    </binding>  
  
    <!--Large tile is an aggregate of multiple stories-->  
    <binding  
      template="TileLarge"  
      arguments="action=storiesClicked&story=43f939ag&story=201c9b1&  
amp;story=d9481ca">  
  
      <text>Can your dog understand what you're saying?</text>  
      ... (another story)  
      ... (one more story)  
  
    </binding>  
  
  </visual>  
</tile>
```

Related articles

- [LaunchActivatedEventArgs.TileActivatedInfo](#) property
- [TileActivatedInfo](#) class

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Primary tile APIs

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Primary tile APIs let you check whether your app is currently pinned to Start, and request to pin your app's primary tile.

ⓘ Important

Requires Creators Update: You must target SDK 15063 and be running Windows 10 build 15063 or later to use the primary tile APIs.

Important APIs: [StartScreenManager class](#), [ContainsAppListEntryAsync](#), [RequestAddAppListEntryAsync](#)

When to use primary tile APIs

You put a lot of effort into designing a great experience for your app's primary tile, and now you have the opportunity to ask the user to pin it to Start. But before we dive into the code, here are some things to keep in mind as you're designing your experience:

- **Do** craft a non-disruptive and easily dismissible UX in your app with a clear "Pin Live Tile" call to action.
- **Do** clearly explain the value of your app's Live Tile before asking the user to pin it.
- **Don't** ask a user to pin your app's tile if the tile is already pinned or the device doesn't support it (more info follows).
- **Don't** repeatedly ask the user to pin your app's tile (they will probably get annoyed).
- **Don't** call the pin API without explicit user interaction or when your app is minimized/not open.

Checking whether the API's exist

If your app supports older versions of Windows 10, you need to check whether these primary tile APIs are available. You do this by using `ApiInformation`. If the primary tile APIs aren't available, avoid executing any calls to the APIs.

C#

```
if
(ApiInformation.IsTypePresent("Windows.UI.StartScreen.StartScreenManager"))
{
    // Primary tile API's supported!
}
else
{
    // Older version of Windows, no primary tile API's
}
```

Check if Start supports your app

Depending on the current Start menu, and your type of app, pinning your app to the current Start screen might not be supported. For example, IoT or xbox devices do not support pinning to Start. Therefore, before showing any pin UI or executing any pin code, you first need to check if your app is even supported for the current Start screen. If it's not supported, don't prompt the user to pin the tile.

C#

```
// Get your own app list entry
// (which is always the first app list entry assuming you are not a multi-
// app package)
AppListEntry entry = (await Package.Current.GetAppListEntriesAsync())[0];

// Check if Start supports your app
bool isSupported =
StartScreenManager.GetDefault().SupportsAppListEntry(entry);
```

Check whether you're currently pinned

To find out if your primary tile is currently pinned to Start, use the `ContainsAppListEntryAsync` method.

C#

```
// Get your own app list entry
AppListEntry entry = (await Package.Current.GetAppListEntriesAsync())[0];
```

```
// Check if your app is currently pinned
bool isPinned = await
StartScreenManager.GetDefault().ContainsAppListEntryAsync(entry);
```

Pin your primary tile

If your primary tile currently isn't pinned, and your tile is supported by Start, you might want to show a tip to users that they can pin your primary tile.

ⓘ Note

You must call this API from a UI thread while your app is in the foreground, and you should only call this API after the user has intentionally requested the primary tile be pinned (for example, after the user clicked yes to your tip about pinning the tile).

If the user clicks your button to pin the primary tile, you call the [RequestAddAppListEntryAsync](#) method to request that your tile be pinned to Start. This will display a dialog asking the user to confirm that they want your tile pinned to Start.

This will return a boolean representing whether your tile is now pinned to Start. If your tile was already pinned, this will immediately return `true` without showing the dialog to the user. If the user clicks "No" on the dialog, or pinning your tile to Start isn't supported, this will return `false`. Otherwise, the user clicked "Yes" and the tile was pinned, and the API will return `true`.

C#

```
// Get your own app list entry
AppListEntry entry = (await Package.Current.GetAppListEntriesAsync())[0];

// And pin it to Start
bool isPinned = await
StartScreenManager.GetDefault().RequestAddAppListEntryAsync(entry);
```

Resources

- [Full code sample on GitHub](#) [↗](#)
- [Pin to taskbar](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Tile and toast notification support for language, scale, and high contrast

Article • 08/08/2024

ⓘ Note

Lives Tiles are a Windows 10 feature that is not supported on later versions of Windows. For new apps, we recommend that you follow the current guidance for [App icons](#).

Your tiles and toasts can load strings and images tailored for display language, [display scale factor](#), high contrast, and other runtime contexts. For background on how to use qualifiers in the names of your resource files, see [Tailor your resources for language, scale, and other qualifiers](#) and [App icons](#).

For more info about the value proposition of localizing your app, see [Globalization and localization](#).

Refer to a string resource from a template

In your tile or toast template, you can refer to a string resource using the `ms-resource` URI (Uniform Resource Identifier) scheme followed by a simple string resource identifier. For example, if you have a Resources.resx file that contains a resource entry whose name is "Farewell", then you have a string resource with the identifier "Farewell". For more info on string resource identifiers and Resources Files (.resw), see [Localize strings in your UI and app package manifest](#).

This is how a reference to the "Farewell" string resource identifier would look in the [text](#) body of your template content, using `ms-resource`.

XML

```
<text id="1">ms-resource:Farewell</text>
```

If you omit the `ms-resource` URI scheme, then the text body is just a string literal, and *not* a reference to an identifier.

XML

```
<text id="1">Farewell</text>
```

Refer to an image resource from a template

In your tile or toast template, you can refer to an image resource using the `ms-appx` URI (Uniform Resource Identifier) scheme followed by the name of the image resource. This is the same way that you refer to an image resource in XAML markup (for more details, see [Reference an image or other asset from XAML markup and code](#)).

For example, you might name folders like this.

```
\Assets\Images\contrast-standard\welcome.png  
\Assets\Images\contrast-high\welcome.png
```

In that case, you have a single image resource and its name (as an absolute path) is `/Assets/Images/welcome.png`. Here's how you use that name in your template.

XML

```
<image id="1" src="ms-appx:///Assets/Images/welcome.png"/>
```

Notice how in this example URI the scheme ("`ms-appx`") is followed by "`://`" which is followed by an absolute path (an absolute path begins with "`/`").

Hosting and loading images in the cloud

The `ms-resource` and `ms-appx` URI schemes perform automatic qualifier matching to find the resource that's most appropriate for the current context. Web URI schemes (for example, `http`, `https`, and `ftp`) do not perform any such automatic matching.

Instead, append onto your image's URI a query string describing the requested qualifier value or values.

XML

```
<image id="1" src="http://www.contoso.com/Assets/Images/welcome.png?ms-  
lang=en-US"/>
```

Then, in the app service that provides your images, implement an HTTP handler that inspects and uses the query string to determine which image to return.

You also need to set the `addImageQuery` attribute to `true` in the `tile` or `toast` notification XML payload. The `addImageQuery` attribute appears in the `visual`, `binding`, and `image` elements of both the tile and toast schemas. Explicitly setting `addImageQuery` on an element overrides any value set on an ancestor. For instance, an `addImageQuery` value of `true` in an `image` element overrides an `addImageQuery` of `false` in its parent `binding` element.

These are the query strings you can use.

[Expand table](#)

Qualifier	Query string	Example
Scale	ms-scale	?ms-scale=400
Language	ms-lang	?ms-lang=en-US
Contrast	ms-contrast	?ms-contrast=high

For a reference table of all the possible qualifier values that you can use in your query strings, see [ResourceContext.QualifierValues](#).

Important APIs

- [ResourceContext.QualifierValues](#)

Related topics

- [Screen sizes and break points for responsive design](#)
 - [Tailor your resources for language, scale, and other qualifiers](#)
 - [App icons.](#)
 - [Globalization and localization](#)
 - [Localize strings in your UI and app package manifest](#)
 - [Reference an image or other asset from XAML markup and code](#)
 - [addImageQuery](#)
 - [Tile schema](#)
 - [Toast schema](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Splash screens

Article • 10/20/2022

All UWP apps must have a splash screen, which is a composite of an image and a background color, both of which can be customized.

Your splash screen is displayed immediately when the user launches your app. This provides immediate feedback to users while app resources are initialized. As soon as your app is ready for interaction, the splash screen is dismissed.

A well-designed splash screen can make your app more inviting. Here's a simple, understated splash screen:



This splash screen is created by combining a green background color with a transparent-background PNG image.

A simple image with a background color looks good regardless of the device your app is running on. Only the size of the background changes to compensate for a variety of screen sizes. Your image always remains intact.

Additionally, you can use the [SplashScreen](#) class to customize your app's launch experience. You can position an extended splash screen, which you create, to give your app more time to complete additional tasks like preparing app UI or completing networking operations. You can also use the **SplashScreen** class to notify you when the splash screen is dismissed, so that you can begin entrance animations.

Topic	Description
-------	-------------

Topic	Description
Add a splash screen	Set your app's splash screen image and background color.
Display a splash screen for more time	Display a splash screen for more time by creating an extended splash screen for your app. This extended screen imitates the splash screen shown when your app is launched, and can be customized.

Add a splash screen

Article • 10/20/2022

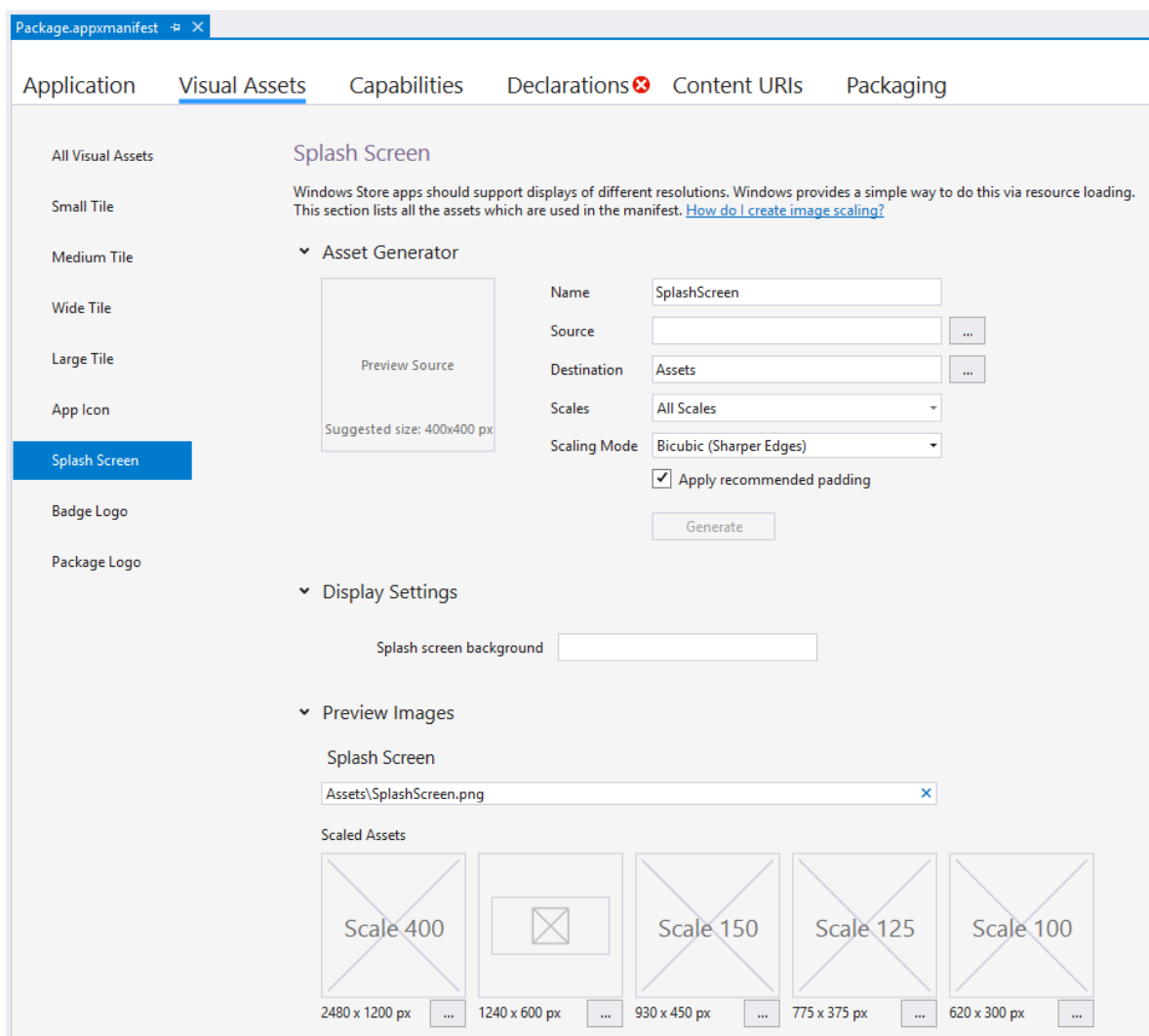
Set your app's splash screen image and background color using Microsoft Visual Studio.

Set the splash screen image and background color in Visual Studio

When you use a Visual Studio template to create your app, a default image is added to your project and set as the splash screen image. The background color for your splash screen defaults to a light gray. If you want to change the default image or color of your app's splash screen, follow these steps:

1. Open your existing Universal Windows Platform (UWP) app project in Visual Studio.
2. From **Solution Explorer**, open the "Package.appxmanifest" file. You can also open this file from the menu bar by choosing **Project > Store > Edit App Manifest**.
3. Open the **Visual Assets** tab and select **Splash Screen** from the **All Visual Assets** pane on the left side of the "Package.appxmanifest" window. If you're changing your splash screen for the first time, you'll see the "Assets\SplashScreen.png" path in the **Splash Screen** field.

The following screen shot shows the "Package.appxmanifest" window in Visual Studio. Depending on the type of project, you will see a slightly different set of visual assets.



If you open "Package.appxmanifest" in a text editor, the **SplashScreen element** appears as a child of the **VisualElements element**. The default splash screen markup in the manifest file looks like this in a text editor:

XML

```
<uap:SplashScreen Image="Assets\SplashScreen.png" />
```

4. To select a new splash screen image for a UWP app, press the button with an ellipsis that appears next to the **1240 x 600 px** label below **Scaled Assets**. Choose the 1240 x 600 pixel image (.png, .jpg, or .jpeg) you'd like to use for your splash screen image.

Important The splash screen image you choose must be 620 x 300 pixels using a 1x scaling factor. Also, when designing your splash screen, note that it is smaller than the screen, and centered. It does not fill the screen like a splash screen for a Windows Phone Store app does.

5. To select a new splash screen image for a Windows Phone Store app, press the button with an ellipsis that appears next to the **1152 x 1920 px** label below **Scaled**

Assets. Choose the 1152 x 1920 pixel image (.png, .jpg, or .jpeg) you'd like to use for your splash screen image.

Important The splash screen image you choose must be 1152 x 1920 pixels which is the correct size for a 2.4x scaling factor. If this is the only asset you provide then it will be scaled down for 1.4x and 1x scaling factors.

6. In the **Background Color** field of the **Splash Screen** section, set the background color displayed with your splash screen image. You can enter either the name of a color or '#' and the hex value of a color. For a list of the names of available colors, see [SplashScreen element](#). Setting a background color for your splash screen is optional. If you don't specify a color for a UWP app, the splash screen background color defaults to a light gray (hex value #464646). This is the same color as the default **Tile** background color (see the **Background Color** field of the **Tile Images and Logos** section in the **Visual Assets** tab). If you don't specify a color for a Windows Phone, or set it to "transparent", then the splash screen background color will be transparent.

Summary and next steps

If your app takes a while to load, consider adding an extended splash screen. For step-by-step guidance, see [Create a customized splash screen](#).

Related topics

- [Create a customized splash screen](#)
- [Package manifest schema reference: SplashScreen element](#)
- [Windows.ApplicationModel.Activation.SplashScreen class](#)

Display a splash screen for more time

Article • 10/20/2022

Display a splash screen for more time by creating an extended splash screen for your app. This extended screen imitates the splash screen shown when your app is launched, but can be customized. Whether you want to show real-time loading information or simply give your app extra time to prepare its initial UI, an extended splash screen lets you define the launch experience.

ⓘ Note

The phrase "extended splash screen" in this topic refers to a splash screen that stays on the screen for an extended period of time. It does not mean a subclass that derives from the [SplashScreen](#) class.

Important APIs

The following APIs are used in this topic:

- [SplashScreen class](#)
- [Window.SizeChanged event](#)
- [Application.OnLaunched method](#)

Default splash screen recommendations

Make sure your extended splash screen accurately imitates the default splash screen by following these recommendations:

- Your extended splash screen page should use a 620 x 300 pixel image that is consistent with the image specified for your splash screen in your app manifest (your app's splash screen image). In Microsoft Visual Studio, splash screen settings are stored in the **Splash Screen** section of the **Visual Assets** tab in your app manifest (Package.appxmanifest file).
- Your extended splash screen should use a background color that is consistent with the background color specified for your splash screen in your app manifest (your app's splash screen background).
- Your code should use the [SplashScreen](#) class to position your app's splash screen image at the same screen coordinates as the default splash screen.

- Your code should respond to window resize events (such as when the screen is rotated or your app is moved next to another app onscreen) by using the [SplashScreen](#) class to reposition items on your extended splash screen.

Use the following steps to create an extended splash screen that effectively imitates the default splash screen.

Add a Blank Page item to your existing app

This topic assumes you want to add an extended splash screen to an existing Universal Windows Platform (UWP) app project using C#, Visual Basic, or C++.

- Open your app in Visual Studio.
- Press or open **Project** from the menu bar and click **Add New Item**. An **Add New Item** dialog box will appear.
- From this dialog box, add a new **Blank Page** to your app. This topic names the extended splash screen page "ExtendedSplash".

Adding a **Blank Page** item generates two files, one for markup (ExtendedSplash.xaml) and another for code (ExtendedSplash.xaml.cs).

Essential XAML for an extended splash screen

Follow these steps to add an image and progress control to your extended splash screen.

In your ExtendedSplash.xaml file:

- Change the [Background](#) property of the default [Grid](#) element to match the background color you set for your app's splash screen in your app manifest (in the **Visual Assets** section of your Package.appxmanifest file). The default splash screen color is a light gray (hex value #464646). Note that this **Grid** element is provided by default when you create a new **Blank Page**. You don't have to use a **Grid**; it's just a convenient base for building an extended splash screen.
- Add a [Canvas](#) element to the [Grid](#). You'll use this **Canvas** to position your extended splash screen image.
- Add an [Image](#) element to the [Canvas](#). Use the same 620 x 300 pixel image for your extended splash screen that you chose for the default splash screen.
- (Optional) Add a progress control to show users that your app is loading. This topic adds a [ProgressRing](#), instead of a determinate or indeterminate [ProgressBar](#).

The following example demonstrates a [Grid](#) with these additions and changes.

XAML

```
<Grid Background="#464646">
    <Canvas>
        <Image x:Name="extendedSplashImage"
Source="Assets/SplashScreen.png"/>
        <ProgressRing Name="splashProgressRing" IsActive="True"
Width="20" HorizontalAlignment="Center"></ProgressRing>
    </Canvas>
</Grid>
```

📌 Note

This example sets the width of the [ProgressRing](#) to 20 pixels. You can manually set its width to a value that works for your app, however, the control will not render at widths of less than 20 pixels.

Essential code for an extended splash screen class

Your extended splash screen needs to respond whenever the window size (Windows only) or orientation changes. The position of the image you use must be updated so that your extended splash screen looks good no matter how the window changes.

Use these steps to define methods to correctly display your extended splash screen.

1. Add required namespaces

You'll need to add the following namespaces to **ExtendedSplash.xaml.cs** to access the [SplashScreen](#) class, the [Rect](#) struct, and the [Window.SizeChanged](#) events.

CS

```
using Windows.ApplicationModel.Activation;
using Windows.Foundation;
using Windows.UI.Core;
```

2. Create a partial class and declare class variables

Include the following code in **ExtendedSplash.xaml.cs** to create a partial class to represent an extended splash screen.

CS

```

partial class ExtendedSplash : Page
{
    internal Rect splashImageRect; // Rect to store splash screen image
    coordinates.
    private SplashScreen splash; // Variable to hold the splash screen
    object.
    internal bool dismissed = false; // Variable to track splash screen
    dismissal status.
    internal Frame rootFrame;

    // Define methods and constructor
}

```

These class variables are used by several methods. The `splashImageRect` variable stores the coordinates where the system displayed the splash screen image for the app. The `splash` variable stores a `SplashScreen` object, and the `dismissed` variable tracks whether or not the splash screen that is displayed by the system has been dismissed.

3. Define a constructor for your class that correctly positions the image

The following code defines a constructor for the extended splash screen class that listens for window resizing events, positions the image and (optional) progress control on the extended splash screen, creates a frame for navigation, and calls an asynchronous method to restore a saved session state.

```

CS

public ExtendedSplash(SplashScreen splashscreen, bool loadState)
{
    InitializeComponent();

    // Listen for window resize events to reposition the extended
    splash screen image accordingly.
    // This ensures that the extended splash screen formats properly in
    response to window resizing.
    Window.Current.SizeChanged += new
    WindowSizeChangedEventArgs(ExtendedSplash_OnResize);

    splash = splashscreen;
    if (splash != null)
    {
        // Register an event handler to be executed when the splash
        screen has been dismissed.
        splash.Dismissed += new TypedEventHandler<SplashScreen, Object>
        (DismissedEventHandler);

        // Retrieve the window coordinates of the splash screen image.
        splashImageRect = splash.ImageLocation;
    }
}

```

```

        PositionImage();

        // If applicable, include a method for positioning a progress
        control.
        PositionRing();
    }

    // Create a Frame to act as the navigation context
    rootFrame = new Frame();
}

```

Make sure to register your [Window.SizeChanged](#) handler (ExtendedSplash_OnResize in the example) in your class constructor so that your app positions the image correctly in your extended splash screen.

4. Define a class method to position the image in your extended splash screen

This code demonstrates how to position the image on the extended splash screen page with the `splashImageRect` class variable.

```

CS

void PositionImage()
{
    extendedSplashImage.SetValue(Canvas.LeftProperty,
    splashImageRect.X);
    extendedSplashImage.SetValue(Canvas.TopProperty,
    splashImageRect.Y);
    extendedSplashImage.Height = splashImageRect.Height;
    extendedSplashImage.Width = splashImageRect.Width;
}

```

5. (Optional) Define a class method to position a progress control in your extended splash screen

If you chose to add a [ProgressRing](#) to your extended splash screen, position it relative to the splash screen image. Add the following code to ExtendedSplash.xaml.cs to center the **ProgressRing** 32 pixels below the image.

```

CS

void PositionRing()
{
    splashProgressRing.SetValue(Canvas.LeftProperty, splashImageRect.X
    + (splashImageRect.Width*0.5) - (splashProgressRing.Width*0.5));
    splashProgressRing.SetValue(Canvas.TopProperty, (splashImageRect.Y
    + splashImageRect.Height + splashImageRect.Height*0.1));
}

```

6. Inside the class, define a handler for the Dismissed event

In `ExtendedSplash.xaml.cs`, respond when the `SplashScreen.Dismissed` event occurs by setting the `dismissed` class variable to `true`. If your app has setup operations, add them to this event handler.

```
CS

// Include code to be executed when the system has transitioned from
// the splash screen to the extended splash screen (application's first
// view).
void DismissedEventHandler(SplashScreen sender, object e)
{
    dismissed = true;

    // Complete app setup operations here...
}
```

After app setup is complete, navigate away from your extended splash screen. The following code defines a method called `DismissExtendedSplash` that navigates to the `MainPage` defined in your app's `MainPage.xaml` file.

```
CS

async void DismissExtendedSplash()
{
    await
Windows.ApplicationModel.Core.CoreApplication.MainView.CoreWindow.Dispa
tcher.RunAsync(CoreDispatcherPriority.Normal, () => {
    rootFrame = new Frame();
    rootFrame.Content = new MainPage(); Window.Current.Content =
rootFrame;
    });
}
```

7. Inside the class, define a handler for Window.SizeChanged events

Prepare your extended splash screen to reposition its elements if a user resizes the window. This code responds when a `Window.SizeChanged` event occurs by capturing the new coordinates and repositioning the image. If you added a progress control to your extended splash screen, reposition it inside this event handler as well.

```
CS

void ExtendedSplash_OnResize(Object sender, WindowSizeChangedEventArgs
e)
```

```

{
    // Safely update the extended splash screen image coordinates. This
    // function will be executed when a user resizes the window.
    if (splash != null)
    {
        // Update the coordinates of the splash screen image.
        splashImageRect = splash.ImageLocation;
        PositionImage();

        // If applicable, include a method for positioning a progress
        // control.
        // PositionRing();
    }
}

```

⚠ Note

Before you try to get the image location make sure the class variable (`splash`) contains a valid [SplashScreen](#) object, as shown in the example.

8. (Optional) Add a class method to restore a saved session state

The code you added to the [OnLaunched](#) method in Step 4: [Modify the launch activation handler](#) causes your app to display an extended splash screen when it launches. To consolidate all methods related to app launch in your extended splash screen class, you could consider adding a method to your `ExtendedSplash.xaml.cs` file to restore the app's state.

```

cs

void RestoreState(bool loadState)
{
    if (loadState)
    {
        // code to load your app's state here
    }
}

```

When you modify the launch activation handler in `App.xaml.cs`, you'll also set `loadstate` to true if the previous [ApplicationExecutionState](#) of your app was **Terminated**. If so, the `RestoreState` method restores the app to its previous state. For an overview of app launch, suspension, and termination, see [App lifecycle](#).

Modify the launch activation handler

When your app is launched, the system passes splash screen information to the app's launch activation event handler. You can use this information to correctly position the image on your extended splash screen page. You can get this splash screen information from the activation event arguments that are passed to your app's [OnLaunched](#) handler (see the `args` variable in the following code).

If you have not already overridden the [OnLaunched](#) handler for your app, see [App lifecycle](#) to learn how to handle activation events.

In `App.xaml.cs`, add the following code to create and display an extended splash screen.

cs

```
protected override void OnLaunched(LaunchActivatedEventArgs args)
{
    if (args.PreviousExecutionState != ApplicationExecutionState.Running)
    {
        bool loadState = (args.PreviousExecutionState ==
ApplicationExecutionState.Terminated);
        ExtendedSplash extendedSplash = new
ExtendedSplash(args.SplashScreen, loadState);
        Window.Current.Content = extendedSplash;
    }

    Window.Current.Activate();
}
```

Complete code

The following code slightly differs from the snippets shown in the previous steps.

- `ExtendedSplash.xaml` includes a `DismissSplash` button. When this button is clicked, an event handler, `DismissSplashButton_Click`, calls the `DismissExtendedSplash` method. In your app, call `DismissExtendedSplash` when your app is done loading resources or initializing its UI.
- This app also uses a UWP app project template, which uses [Frame](#) navigation. As a result, in `App.xaml.cs`, the launch activation handler ([OnLaunched](#)) defines a `rootFrame` and uses it to set the content of the app window.

ExtendedSplash.xaml

This example includes a `DismissSplash` button because it doesn't have app resources to load. In your app, dismiss the extended splash screen automatically when your app is done loading resources or preparing its initial UI.

XML

```
<Page
    x:Class="SplashScreenExample.ExtendedSplash"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="using:SplashScreenExample"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    mc:Ignorable="d">

    <Grid Background="#464646">
        <Canvas>
            <Image x:Name="extendedSplashImage"
Source="Assets/SplashScreen.png"/>
            <ProgressRing Name="splashProgressRing" IsActive="True"
Width="20" HorizontalAlignment="Center"/>
        </Canvas>
        <StackPanel HorizontalAlignment="Center" VerticalAlignment="Bottom">
            <Button x:Name="DismissSplash" Content="Dismiss extended splash
screen" HorizontalAlignment="Center" Click="DismissSplashButton_Click" />
        </StackPanel>
    </Grid>
</Page>
```

ExtendedSplash.xaml.cs

Note that the `DismissExtendedSplash` method is called from the click event handler for the `DismissSplash` button. In your app, you won't need a `DismissSplash` button. Instead, call `DismissExtendedSplash` when your app is done loading resources and you want to navigate to its main page.

CS

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.InteropServices.WindowsRuntime;
using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
```

```

using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;

using Windows.ApplicationModel.Activation;
using SplashScreenExample.Common;
using Windows.UI.Core;

// The Blank Page item template is documented at
https://go.microsoft.com/fwlink/p/?LinkID=234238

namespace SplashScreenExample
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
    Frame.
    /// </summary>
    partial class ExtendedSplash : Page
    {
        internal Rect splashImageRect; // Rect to store splash screen image
        coordinates.
        private SplashScreen splash; // Variable to hold the splash screen
        object.
        internal bool dismissed = false; // Variable to track splash screen
        dismissal status.
        internal Frame rootFrame;

        public ExtendedSplash(SplashScreen splashscreen, bool loadState)
        {
            InitializeComponent();

            // Listen for window resize events to reposition the extended
            splash screen image accordingly.
            // This is important to ensure that the extended splash screen
            is formatted properly in response to snapping, unsnapping, rotation, etc...
            Window.Current.SizeChanged += new
            WindowSizeChangedEventHandler(ExtendedSplash_OnResize);

            splash = splashscreen;

            if (splash != null)
            {
                // Register an event handler to be executed when the splash
                screen has been dismissed.
                splash.Dismissed += new TypedEventHandler<SplashScreen,
                Object>(DismissedEventHandler);

                // Retrieve the window coordinates of the splash screen
                image.
                splashImageRect = splash.ImageLocation;
                PositionImage();

                // Optional: Add a progress ring to your splash screen to
                show users that content is loading
                PositionRing();
            }
        }
    }
}

```

```

    }

    // Create a Frame to act as the navigation context
    rootFrame = new Frame();

    // Restore the saved session state if necessary
    RestoreState(loadState);
}

void RestoreState(bool loadState)
{
    if (loadState)
    {
        // TODO: write code to load state
    }
}

// Position the extended splash screen image in the same location as
the system splash screen image.
void PositionImage()
{
    extendedSplashImage.SetValue(Canvas.LeftProperty,
splashImageRect.X);
    extendedSplashImage.SetValue(Canvas.TopProperty,
splashImageRect.Y);
    extendedSplashImage.Height = splashImageRect.Height;
    extendedSplashImage.Width = splashImageRect.Width;
}

void PositionRing()
{
    splashProgressRing.SetValue(Canvas.LeftProperty,
splashImageRect.X + (splashImageRect.Width*0.5) -
(splashProgressRing.Width*0.5));
    splashProgressRing.SetValue(Canvas.TopProperty,
(splashImageRect.Y + splashImageRect.Height + splashImageRect.Height*0.1));
}

void ExtendedSplash_OnResize(Object sender,
WindowSizeChangedEventArgs e)
{
    // Safely update the extended splash screen image coordinates.
    This function will be fired in response to snapping, unsnapping, rotation,
    etc...

    if (splash != null)
    {
        // Update the coordinates of the splash screen image.
        splashImageRect = splash.ImageLocation;
        PositionImage();
        PositionRing();
    }
}

// Include code to be executed when the system has transitioned from

```

```

the splash screen to the extended splash screen (application's first view).
    void DismissedEventHandler(SplashScreen sender, object e)
    {
        dismissed = true;

        // Complete app setup operations here...
    }

    void DismissExtendedSplash()
    {
        // Navigate to mainpage
        rootFrame.Navigate(typeof(MainPage));
        // Place the frame in the current Window
        Window.Current.Content = rootFrame;
    }

    void DismissSplashButton_Click(object sender, RoutedEventArgs e)
    {
        DismissExtendedSplash();
    }
}
}

```

App.xaml.cs

This project was created using the UWP app **Blank App (XAML)** project template in Visual Studio. Both the `OnNavigationFailed` and `OnSuspending` event handlers are automatically generated and don't need to be changed to implement an extended splash screen. This topic only modifies `OnLaunched`.

If you didn't use a project template for your app, see Step 4: [Modify the launch activation handler](#) for an example of a modified `OnLaunched` that doesn't use `Frame` navigation.

CS

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.InteropServices.WindowsRuntime;
using Windows.ApplicationModel;
using Windows.ApplicationModel.Activation;
using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;

```

```

using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;

// The Blank Application template is documented at
https://go.microsoft.com/fwlink/p/?LinkID=234227

namespace SplashScreenExample
{
    /// <summary>
    /// Provides application-specific behavior to supplement the default
    Application class.
    /// </summary>
    sealed partial class App : Application
    {
        /// <summary>
        /// Initializes the singleton application object. This is the first
        line of authored code
        /// executed, and as such is the logical equivalent of main() or
        WinMain().
        /// </summary>
        public App()
        {
            Microsoft.ApplicationInsights.WindowsAppInitializer.InitializeAsync(
                Microsoft.ApplicationInsights.WindowsCollectors.Metadata |
                Microsoft.ApplicationInsights.WindowsCollectors.Session);
            this.InitializeComponent();
            this.Suspending += OnSuspending;
        }

        /// <summary>
        /// Invoked when the application is launched normally by the end
        user. Other entry points
        /// will be used such as when the application is launched to open a
        specific file.
        /// </summary>
        /// <param name="e">Details about the launch request and process.
        </param>
        protected override void OnLaunched(LaunchActivatedEventArgs e)
        {
            #if DEBUG
                if (System.Diagnostics.Debugger.IsAttached)
                {
                    this.DebugSettings.EnableFrameRateCounter = true;
                }
            #endif

            Frame rootFrame = Window.Current.Content as Frame;

            // Do not repeat app initialization when the Window already has
            content,

            // just ensure that the window is active
            if (rootFrame == null)
            {
                // Create a Frame to act as the navigation context and

```

```

navigate to the first page
    rootFrame = new Frame();
    // Set the default language
    rootFrame.Language =
Windows.Globalization.ApplicationLanguages.Languages[0];

    rootFrame.NavigationFailed += OnNavigationFailed;

    // Display an extended splash screen if app was not
previously running.
    if (e.PreviousExecutionState !=
ApplicationExecutionState.Running)
    {
        bool loadState = (e.PreviousExecutionState ==
ApplicationExecutionState.Terminated);
        ExtendedSplash extendedSplash = new
ExtendedSplash(e.SplashScreen, loadState);
        rootFrame.Content = extendedSplash;
        Window.Current.Content = rootFrame;
    }
}

if (rootFrame.Content == null)
{
    // When the navigation stack isn't restored navigate to the
first page,
    // configuring the new page by passing required information
as a navigation
    // parameter
    rootFrame.Navigate(typeof(MainPage), e.Arguments);
}
// Ensure the current window is active
Window.Current.Activate();
}

/// <summary>
/// Invoked when Navigation to a certain page fails
/// </summary>
/// <param name="sender">The Frame which failed navigation</param>
/// <param name="e">Details about the navigation failure</param>
void OnNavigationFailed(object sender, NavigationFailedEventArgs e)
{
    throw new Exception("Failed to load Page " +
e.SourcePageType.FullName);
}

/// <summary>
/// Invoked when application execution is being suspended.
Application state is saved
/// without knowing whether the application will be terminated or
resumed with the contents
/// of memory still intact.
/// </summary>
/// <param name="sender">The source of the suspend request.</param>
/// <param name="e">Details about the suspend request.</param>

```

```
private void OnSuspending(object sender, SuspendingEventArgs e)
{
    var deferral = e.SuspendingOperation.GetDeferral();
    // TODO: Save application state and stop any background activity
    deferral.Complete();
}
}
```

Related topics

- [App lifecycle](#)

Reference

- [Windows.ApplicationModel.Activation namespace](#)
- [Windows.ApplicationModel.Activation.SplashScreen class](#)
- [Windows.ApplicationModel.Activation.SplashScreen.ImageLocation property](#)
- [Windows.ApplicationModel.Core.CoreApplicationView.Activated event](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Launch an app with a URI

Article • 12/06/2024

This section describes how to use a Uniform Resource Identifier (URI) to launch one app from another app, enabling helpful app-to-app scenarios.

 Expand table

Topic	Description
Launch the default app for a URI	Learn how to launch the default app for a Uniform Resource Identifier (URI). URIs allow you to launch another app to perform a specific task. This topic also provides an overview of the many URI schemes built into Windows.
Handle URI activation	Learn how to register an app to become the default handler for a Uniform Resource Identifier (URI) scheme name.
Launch an app for results	Learn how to launch an app from another app and exchange data between the two. This is called launching an app for results.
Choose and save tones using the ms-tonepicker URI scheme	This topic describes the ms-tonepicker URI scheme and how to use it to display a tone picker to select a tone, save a tone, and get the friendly name for a tone.
Launch the Windows Settings app	Learn how to launch the Windows Settings app from your app. This topic describes the ms-settings URI scheme. Use this URI scheme to launch the Windows Settings app to specific settings pages.
Launch the UWP app	This topic describes the ms-windows-store URI scheme. Your app can use this URI scheme to launch the UWP app to specific pages in the Store.
Launch the Windows Maps app	Learn how to launch the Windows Maps app from your app.
Launch the People app	This topic describes the ms-people URI scheme. Your app can use this URI scheme to launch the People app for specific actions.
Launch screen snipping	Learn how to use URI schemes to open a new snip, or to open the Snip & Sketch app.
Enable apps for websites using app URI handlers	Drive user engagement with your app by supporting the Apps for Websites feature.

Related content

- [Launch an app on a remote device](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Launch the default app for a URI

Article • 10/20/2022

Important APIs

- [LaunchUriAsync](#)
- [PreferredApplicationPackageFamilyName](#)
- [DesiredRemainingView](#)

Learn how to launch the default app for a Uniform Resource Identifier (URI). URIs allow you to launch another app to perform a specific task. This topic also provides an overview of the many URI schemes built into Windows. You can launch custom URIs too. For more info about registering a custom URI scheme and handling URI activation, see [Handle URI activation](#).

URI schemes let you open apps by clicking hyperlinks. Just as you can start a new email using **mailto:**, you can open the default web browser using **http:**

This topic describes the following URI schemes built into Windows:

 Expand table

URI Scheme	Launches
bingmaps: , ms-drive-to: , and ms-walk-to:	Maps app
http: and https:	Default web browser
mailto:	Default email app
microsoft-edge:	Microsoft Edge browser
ms-call:	Call app
ms-chat:	Messaging app
ms-people:	People app
ms-photos:	Photos app
ms-recall:	Click to do (part of Recall)
ms-settings:	Settings app
ms-store:	Store app
ms-tonepicker:	Tone picker

URI Scheme	Launches
ms-yellowpage:	Nearby Numbers app
msnweather:	Weather app

For example, the following URI opens the default browser and displays the Microsoft Bing web site: `https://bing.com/`.

You can also launch custom URI schemes too. If there is no app installed to handle that URI, you can recommend an app for the user to install. For more info, see [Recommend an app if one is not available to handle the URI](#).

In general, your app can't select the app that is launched. The user determines which app is launched. More than one app can register to handle the same URI scheme. The exception to this is for reserved URI schemes. Registrations of reserved URI schemes are ignored. For the full list of reserved URI schemes, see [Handle URI activation](#). In cases where more than one app may have registered the same URI scheme, your app can recommend a specific app to be launched. For more info, see [Recommend an app if one is not available to handle the URI](#).

Call `LaunchUriAsync` to launch a URI

Use the [LaunchUriAsync](#) method to launch a URI. When you call this method, your app must be the foreground app, that is, it must be visible to the user. This requirement helps ensure that the user remains in control. To meet this requirement, make sure that you tie all URI launches directly to the UI of your app. The user must always take some action to initiate a URI launch. If you attempt to launch a URI and your app isn't in the foreground, the launch will fail and your error callback will be invoked.

First create a [System.Uri](#) object to represent the URI, then pass that to the [LaunchUriAsync](#) method. Use the return result to see if the call succeeded, as shown in the following example.

CS

```
private async void launchURI_Click(object sender, RoutedEventArgs e)
{
    // The URI to launch
    var uriBing = new Uri(@"http://www.bing.com");

    // Launch the URI
    var success = await Windows.System.Launcher.LaunchUriAsync(uriBing);

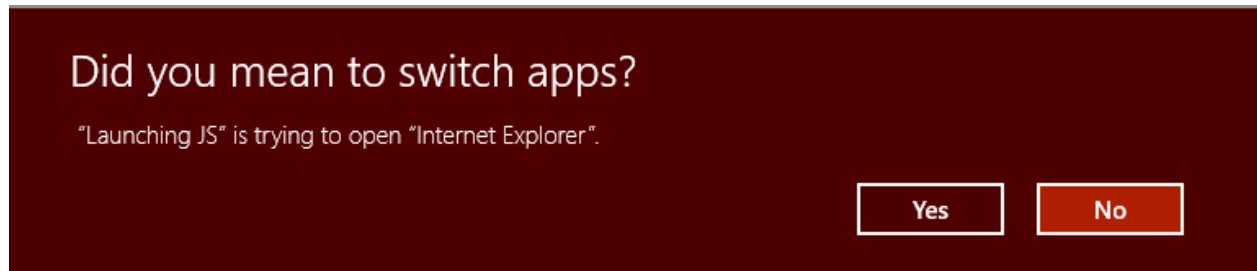
    if (success)
```

```

{
    // URI launched
}
else
{
    // URI launch failed
}
}

```

In some cases, the operating system will prompt the user to see if they actually want to switch apps.



If you always want this prompt to occur, use the [Windows.System.LauncherOptions.TreatAsUntrusted](#) property to tell the operating system to display a warning.

CS

```

// The URI to launch
var uriBing = new Uri(@"http://www.bing.com");

// Set the option to show a warning
var promptOptions = new Windows.System.LauncherOptions();
promptOptions.TreatAsUntrusted = true;

// Launch the URI
var success = await Windows.System.Launcher.LaunchUriAsync(uriBing,
promptOptions);

```

Recommend an app if one is not available to handle the URI

In some cases, the user might not have an app installed to handle the URI that you are launching. By default, the operating system handles these cases by providing the user with a link to search for an appropriate app on the store. If you want to give the user a specific recommendation for which app to acquire in this scenario, you can do so by passing that recommendation along with the URI that you are launching.

Recommendations are also useful when more than one app has registered to handle a URI scheme. By recommending a specific app, Windows will open that app if it is already installed.

To make a recommendation, call the `Windows.System.Launcher.LaunchUriAsync(Uri, LauncherOptions)` method with `LauncherOptions.preferredApplicationPackageFamilyName` set to the package family name of the app in the store that you want to recommend. The operating system uses this info to replace the general option to search for an app in the store with a specific option to acquire the recommended app from the store.

CS

```
// Set the recommended app
var options = new Windows.System.LauncherOptions();
options.PreferredApplicationPackageFamilyName =
    "Contoso.URIApp_8wknc82po1e";
options.PreferredApplicationDisplayName = "Contoso URI Ap";

// Launch the URI and pass in the recommended app
// in case the user has no apps installed to handle the URI
var success = await Windows.System.Launcher.LaunchUriAsync(uriContoso,
    options);
```

Set remaining view preference

Source apps that call `LaunchUriAsync` can request that they remain on screen after a URI launch. By default, Windows attempts to share all available space equally between the source app and the target app that handles the URI. Source apps can use the `DesiredRemainingView` property to indicate to the operating system that they prefer their app window to take up more or less of the available space. `DesiredRemainingView` can also be used to indicate that the source app doesn't need to remain on screen after the URI launch and can be completely replaced by the target app. This property only specifies the preferred window size of the calling app. It doesn't specify the behavior of other apps that may happen to also be on screen at the same time.

Note Windows takes into account multiple different factors when it determines the source app's final window size, for example, the preference of the source app, the number of apps on screen, the screen orientation, and so on. By setting `DesiredRemainingView`, you aren't guaranteed a specific windowing behavior for the source app.

CS

```
// Set the desired remaining view.
var options = new Windows.System.LauncherOptions();
options.DesiredRemainingView =
Windows.UI.ViewManagement.ViewSizePreference.UseLess;

// Launch the URI
var success = await Windows.System.Launcher.LaunchUriAsync(uriContoso,
options);
```

URI Schemes

The various URI schemes are described below.

Call app URI scheme

Use the **ms-call**: URI scheme to launch the Call app.

 Expand table

URI Scheme	Result
<code>ms-call:settings</code>	Calls app settings page.

Email URI scheme

Use the **mailto**: URI scheme to launch the default mail app.

 Expand table

URI Scheme	Results
<code>mailto:</code>	Launches the default email app.
<code>mailto:[email address]</code>	Launches the email app and creates a new message with the specified email address on the To line. Note that the email is not sent until the user taps send.

HTTP URI scheme

Use the **http**: URI scheme to launch the default web browser.

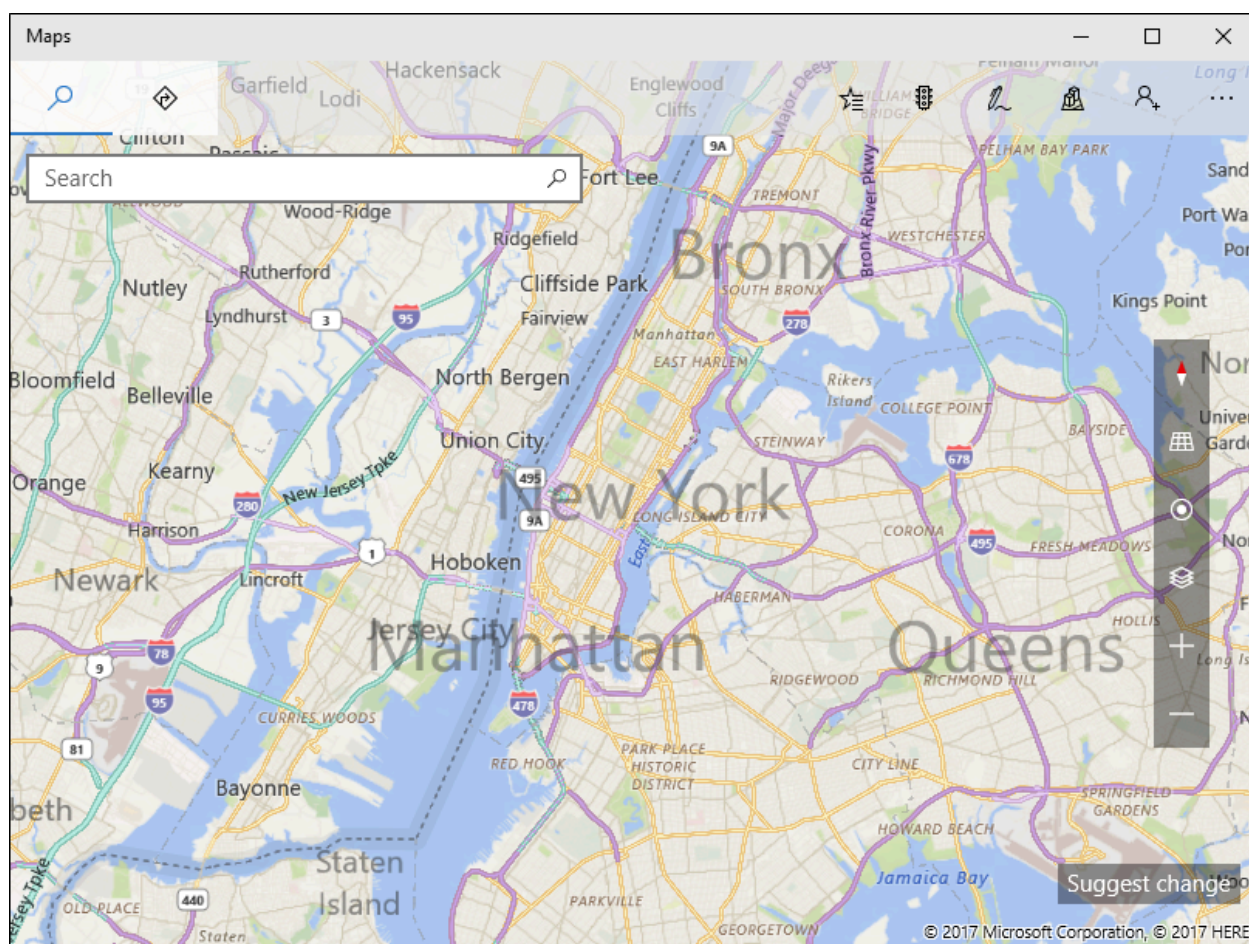
 Expand table

URI Scheme	Results
<code>http:</code>	Launches the default web browser.

Maps app URI schemes

Use the `bingmaps:`, `ms-drive-to:`, and `ms-walk-to:` URI schemes to [launch the Windows Maps app](#) to specific maps, directions, and search results. For example, the following URI opens the Windows Maps app and displays a map centered over New York City.

```
bingmaps:?cp=40.726966~-74.006076
```



For more info, see [Launch the Windows Maps app](#). To use the map control in your own app, see [Display maps with 2D, 3D, and Streetside views](#).

Messaging app URI scheme

Use the `ms-chat:` URI scheme to launch the Windows Messaging app.

URI scheme	Results
<code>ms-chat:</code>	Launches the Messaging app.
<code>ms-chat:?ContactID={contacted}</code>	Allows the messaging application to be launched with a particular contact's information.
<code>ms-chat:?Body={body}</code>	Allows the messaging application to be launched with a string to use as the content of the message.
<code>ms-chat:?Addresses={address}&Body={body}</code>	Allows the messaging application to be launched with a particular addresses' information, and with a string to use as the content of the message. Note: Addresses can be concatenated.
<code>ms-chat:?TransportId={transportId}</code>	Allows the messaging application to be launched with a particular transport ID.

Tone picker URI scheme

Use the `ms-tonepicker:` URI scheme to choose ringtones, alarms, and system tones. You can also save new ringtones and get the display name of a tone.

[Expand table](#)

URI Scheme	Results
<code>ms-tonepicker:</code>	Pick ringtones, alarms, and system tones.

Parameters are passed via a [ValueSet](#) to the LaunchURI API. See [Choose and save tones using the ms-tonepicker URI scheme](#) for details.

Nearby Numbers app URI scheme

Use the `ms-yellowpage:` URI scheme to launch the Nearby Numbers app.

[Expand table](#)

URI Scheme	Results
<code>ms-yellowpage:?input=[keyword]&method=[String or T9]</code>	Launches the Nearby Numbers app. <code>input</code> refers to the keyword you want to search. <code>method</code> refers to the type of search (string or T9 search). If <code>method</code> is <code>T9</code> (a type of keyboard) then <code>keyword</code> should be a numeric string that maps to the T9 keyboard letters to search for.

URI Scheme	Results
	If <code>method</code> is <code>String</code> then <code>keyword</code> is the keyword to search for.

People app URI scheme

Use the `ms-people:` URI scheme to launch the People app. For more info, see [Launch the People app](#).

Photos app URI scheme

Use the `ms-photos:` URI scheme to launch the Photos app to view an image or edit a video. For example:

To view an image: `ms-photos:viewer?fileName=c:\users\userName\Pictures\image.jpg`

Or to edit a video: `ms-photos:videoedit?`

`InputToken=123abc&Action=Trim&StartTime=01:02:03`

ⓘ Note

The URIs to edit a video or display an image are only available on desktop.

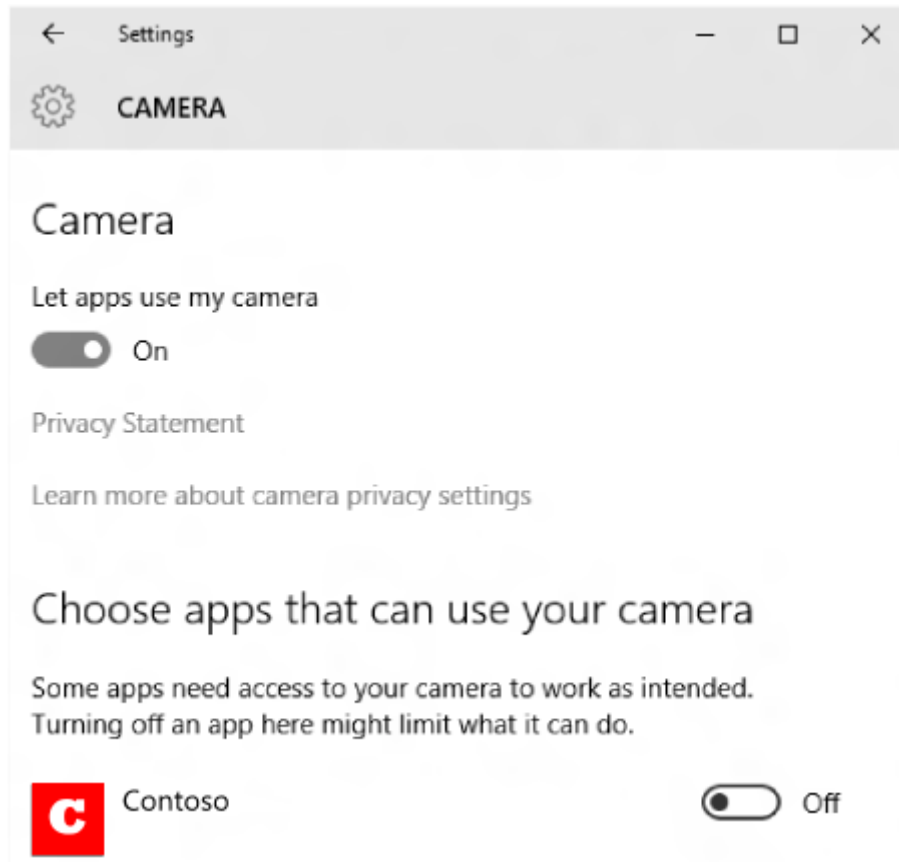
[Expand table](#)

URI scheme	Results
<code>ms-photos:viewer?</code> <code>fileName={filename}</code>	Launches the Photos app to view the specified image where {filename} is a fully-qualified path name. For example: <code>c:\users\userName\Pictures\ImageToView.jpg</code>
<code>ms-photos:videoedit?</code> <code>InputToken={input token}</code>	Launches the Photos app in video editing mode for the file represented by the file token. InputToken is required. Use the SharedStorageAccessManager to get a token for a file.
<code>ms-photos:videoedit?</code> <code>Action={action}</code>	A parameter that indicates which video editing mode to open the Photos app in, where {action} is one of: SlowMotion , FrameExtraction , Trim , View , Ink . Action is required.
<code>ms-photos:videoedit?</code> <code>StartTime={timespan}</code>	An optional parameter that specifies where to start playing the video. {timespan} must be in the format <code>"hh:mm:ss.ffff"</code> . If not specified, defaults to <code>00:00:00.0000</code>

Settings app URI scheme

Use the **ms-settings:** URI scheme to [launch the Windows Settings app](#). Launching to the Settings app is an important part of writing a privacy-aware app. If your app can't access a sensitive resource, we recommend providing the user a convenient link to the privacy settings for that resource. For example, the following URI opens the Settings app and displays the camera privacy settings.

```
ms-settings:privacy-webcam
```



For more info, see [Launch the Windows Settings app](#) and [Guidelines for privacy-aware apps](#).

Store app URI scheme

Use the **ms-windows-store:** URI scheme to [Launch the UWP app](#). Open product detail pages, product review pages, and search pages, etc. For example, the following URI opens the UWP app and launches the home page of the Store.

```
ms-windows-store://home/
```

For more info, see [Launch the UWP app](#).

Weather app URI scheme

Use the **msnweather:** URI scheme to launch the Weather app.

 Expand table

URI Scheme	Results
<code>msnweather://forecast?la=\</code> <code>[latitude\]&lo=\[longitude\]</code>	Launches the Weather app in the Forecast page based on a location geographic coordinates. <code>latitude</code> refers to the latitude of the location. <code>longitude</code> refers to the longitude of the location.

Microsoft Edge URI scheme

Use the **microsoft-edge:** URI scheme to launch the Microsoft Edge browser to a specified URL.

 Expand table

URI Scheme	Results
<code>microsoft-</code> <code>edge:https://example.com/</code>	Opens the Microsoft Edge browser and navigates to <code>https://example.com/</code>

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Handle URI activation

Article • 10/20/2022

Important APIs

- [Windows.ApplicationModel.Activation.ProtocolActivatedEventArgs](#)
- [Windows.UI.Xaml.Application.OnActivated](#)

Learn how to register an app to become the default handler for a Uniform Resource Identifier (URI) scheme name. Both Windows desktop apps and Universal Windows Platform (UWP) apps can register to be a default handler for a URI scheme name. If the user chooses your app as the default handler for a URI scheme name, your app will be activated every time that type of URI is launched.

We recommend that you only register for a URI scheme name if you expect to handle all URI launches for that type of URI scheme. If you do choose to register for a URI scheme name, you must provide the end user with the functionality that is expected when your app is activated for that URI scheme. For example, an app that registers for the mailto: URI scheme name should open to a new e-mail message so that the user can compose a new e-mail. For more info on URI associations, see [Guidelines and checklist for file types and URIs](#).

These steps show how to register for a custom URI scheme name, `alsdk://`, and how to activate your app when the user launches a `alsdk://` URI.

ⓘ Note

In UWP apps, certain URIs and file extensions are reserved for use by built-in apps and the operating system. Attempts to register your app with a reserved URI or file extension will be ignored. See [Reserved URI scheme names and file types](#) for an alphabetic list of Uri schemes that you can't register for your UWP apps because they are either reserved or forbidden.

Step 1: Specify the extension point in the package manifest

The app receives activation events only for the URI scheme names listed in the package manifest. Here's how you indicate that your app handles the `alsdk` URI scheme name.

1. In the **Solution Explorer**, double-click `package.appxmanifest` to open the manifest designer. Select the **Declarations** tab and in the **Available Declarations** drop-down, select **Protocol** and then click **Add**.

Here is a brief description of each of the fields that you may fill in the manifest designer for the Protocol (see [AppX Package Manifest](#) for details):

Field	Description
Logo	Specify the logo that is used to identify the URI scheme name in the Set Default Programs on the Control Panel . If no Logo is specified, the small logo for the app is used.
Display Name	Specify the display name to identify the URI scheme name in the Set Default Programs on the Control Panel .
Name	Choose a name for the Uri scheme.
Note The Name must be in all lower case letters.	
Reserved and forbidden file types See Reserved URI scheme names and file types for an alphabetic list of Uri schemes that you can't register for your UWP apps because they are either reserved or forbidden.	
Executable	Specifies the default launch executable for the protocol. If not specified, the app's executable is used. If specified, the string must be between 1 and 256 characters in length, must end with ".exe", and cannot contain these characters: >, <, :, ", , ?, or *. If specified, the Entry point is also used. If the Entry point isn't specified, the entry point defined for the app is used.
Entry point	Specifies the task that handles the protocol extension. This is normally the fully namespace-qualified name of a Windows Runtime type. If not specified, the entry point for the app is used.
Start page	The web page that handles the extensibility point.
Resource group	A tag that you can use to group extension activations together for resource management purposes.
Desired View (Windows-only)	Specify the Desired View field to indicate the amount of space the app's window needs when it is launched for the URI scheme name. The possible values for Desired View are Default, UseLess, UseHalf, UseMore, or UseMinimum. Note Windows takes into account multiple different factors when determining the target app's final window size, for example, the preference of the source app, the number of apps on screen, the screen orientation, and so on. Setting Desired View doesn't guarantee a specific windowing behavior for the target app. Mobile device family: Desired View isn't supported on the mobile device family.

2. Enter `images\Icon.png` as the **Logo**.

3. Enter `SDK Sample URI Scheme` as the **Display name**
4. Enter `alsdk` as the **Name**.
5. Press Ctrl+S to save the change to package.appxmanifest.

This adds an **Extension** element like this one to the package manifest. The **windows.protocol** category indicates that the app handles the `alsdk` URI scheme name.

XML

```
<Applications>
  <Application Id= ... >
    <Extensions>
      <uap:Extension Category="windows.protocol">
        <uap:Protocol Name="alsdk">
          <uap:Logo>images\icon.png</uap:Logo>
          <uap:DisplayName>SDK Sample URI Scheme</uap:DisplayName>
        </uap:Protocol>
      </uap:Extension>
    </Extensions>
    ...
  </Application>
</Applications>
```

Step 2: Add the proper icons

Apps that become the default for a URI scheme name have their icons displayed in various places throughout the system such as in the Default programs control panel. Include a 44x44 icon with your project for this purpose. Match the look of the app tile logo and use your app's background color rather than making the icon transparent. Have the logo extend to the edge without padding it. Test your icons on white backgrounds. See [App icons and logos](#) for more details about icons.

Step 3: Handle the activated event

The **OnActivated** event handler receives all activation events. The **Kind** property indicates the type of activation event. This example is set up to handle **Protocol** activation events.

C#

```
public partial class App
{
```

```

protected override void OnActivated(IActivatedEventArgs args)
{
    if (args.Kind == ActivationKind.Protocol)
    {
        ProtocolActivatedEventArgs eventArgs = args as
ProtocolActivatedEventArgs;
        // TODO: Handle URI activation
        // The received URI is eventArgs.Uri.AbsoluteUri
    }
}
}

```

VB

```

Protected Overrides Sub OnActivated(ByVal args As
Windows.ApplicationModel.Activation.IActivatedEventArgs)
    If args.Kind = ActivationKind.Protocol Then
        ProtocolActivatedEventArgs eventArgs = args As
ProtocolActivatedEventArgs

        ' TODO: Handle URI activation
        ' The received URI is eventArgs.Uri.AbsoluteUri
    End If
End Sub

```

C++/WinRT

```

void
App::OnActivated(Windows::ApplicationModel::Activation::IActivatedEventArgs
const& args)
{
    if (args.Kind() ==
Windows::ApplicationModel::Activation::ActivationKind::Protocol)
    {
        auto protocolActivatedEventArgs{
args.as<Windows::ApplicationModel::Activation::ProtocolActivatedEventArgs>()
};
        // TODO: Handle URI activation
        auto receivedURI{ protocolActivatedEventArgs.Uri().RawUri() };
    }
}

```

C++

```

void
App::OnActivated(Windows::ApplicationModel::Activation::IActivatedEventArgs^
args)
{
    if (args->Kind ==
Windows::ApplicationModel::Activation::ActivationKind::Protocol)
    {

```

```

        Windows::ApplicationModel::Activation::ProtocolActivatedEventArgs^
eventArgs =

dynamic_cast<Windows::ApplicationModel::Activation::ProtocolActivatedEventAr
gs^>(args);

        // TODO: Handle URI activation
        // The received URI is eventArgs->Uri->RawUri
    }
}

```

ⓘ Note

When launched via Protocol Contract, make sure that Back button takes the user back to the screen that launched the app and not to the app's previous content.

The following code programmatically launches the app via its URI:

```

C#

// Launch the URI
var uri = new Uri("alsdk:");
var success = await Windows.System.Launcher.LaunchUriAsync(uri)

```

For more details about how to launch an app via a URI, see [Launch the default app for a URI](#).

It is recommended that apps create a new XAML **Frame** for each activation event that opens a new page. This way, the navigation backstack for the new XAML **Frame** will not contain any previous content that the app might have on the current window when suspended. Apps that decide to use a single XAML **Frame** for Launch and File Contracts should clear the pages on the **Frame** navigation journal before navigating to a new page.

When launched via Protocol activation, apps should consider including UI that allows the user to go back to the top page of the app.

Remarks

Any app or website can use your URI scheme name, including malicious ones. So any data that you get in the URI could come from an untrusted source. We recommend that you never perform a permanent action based on the parameters that you receive in the